Two mini-talks on continual learning

ML Reading Group

April 24, 2026

1 What is Continual Learning? (Vijay)

The objective of continual learning is to teach models new things without overwriting what they already know: learning without catastrophic forgetting.

1.1 Defining Catastrophic Forgetting

A phenomenon where neural networks experience a sudden, non-gradual performance collapse on previously mastered Task A after being fine-tuned on Task B.

- **How to measure catastrophic forgetting?** Quantified by evaluating the model on Task A, training on Task B, and re-evaluating on Task A. Forgetting is the delta in performance.
- **Causality (Superposition):** While catastrophic forgetting is by no means limited to LLMs (or neural networks), they are particularly susceptible because individual weights often represent multiple features / tasks simultaneously: superposition. So updating a weight for Task B could disrupt its utility for Task A.

2 Mitigation Strategies

2.1 Regularization

Constraining parameter shifts during fine-tuning to stay within a "safe" region of parameter space. Often, this means not straying too far from the reference policy.

- **KL Penalty:** Adding a KL divergence term to the reward function to ensure the new policy π_θ does not drift too far from the reference policy π_{ref}.
- **L2 Penalty:** Constrain the weight update ΔW such that

  $$||\Delta W||_F \leq \epsilon$$

  where $||\cdot||_F$ indicates the frobenius norm (this will show up later!).

- **Fisher Information Matrix (FIM):** A more advanced approach that measures the predictive importance of individual weights. Whereas the L2 penalty implicitly assumes that "all weights are created equal" (insofar as they contribute to the predictive power of the model), the FIM gives us a way to identify and protect "important" parameters, those with high Fisher information.

2.2 Architectural Changes: LoRA

Low-Rank Adaptation (LoRA): Instead of updating the full weight matrix $W \in \mathbb{R}^{n \times d}$, we freeze W and learn a low-rank perturbation $\Delta W = AB$, where $A \in \mathbb{R}^{n \times r}$ and $B \in \mathbb{R}^{r \times d}$ with $r \ll \min(n, d)$. [1] This limits the degrees of freedom for change and has the added benefit of consuming less memory. This also explains why industry fine-tuning tools like **Thinking Machines Tinker API** only support LoRA fine tuning.

2.3 Replay Buffers

An RL-centric strategy where a buffer B stores experiences (states/actions) from older policies. During training on new tasks, the model samples from B to "review" old material, instead of always sampling from the current policy π_θ This is analogous to studying for a cumulative final exam: you practice problems from before the midterm so that you don't forget what you were already tested on.

3 Evolutionary Strategies lead to Catastrophic Forgetting in LLMs (Maggie)

We reviewed a recent paper[2] comparing Evolutionary Strategies (ES) and Group Relative Policy Optimization (GRPO) in the context of LLM post-training.

3.1 Algorithm Mechanics

- **GRPO:** A gradient-based RL algorithm. It samples a group of completions for a prompt, calculates relative advantages within the group, and uses backpropagation to perform targeted weight updates.
- **ES:** A gradient-free, black-box optimizer. It generates N (e.g., 30) copies of the model, each perturbed by random noise ϵ. It evaluates all copies and computes a weighted average of the perturbations based on performance.

Crucially, because there are no gradients and no backprop, the memory & compute footprint of ES is substantially lower than GRPO. However, as we will see, this comes at the cost of incurring catastrophic forgetting.

[1] A fun fact is that this technique does not work if both A, B are initialized to exactly zero.
[2] https://arxiv.org/pdf/2601.20861

3.2 Experimental Results (Countdown vs. HellaSwag)

Models (Qwen-2.5-1.5B, Llama-3.2-1B) were trained on the **Countdown** dataset (arithmetic reasoning) while monitoring the **HellaSwag** benchmark (sentence completion, which the pretrained model should already be good at).

- **Performance:** Both algorithms achieved comparable peak validation accuracy on the new task.
- **Forgetting:** ES exhibited severe catastrophic forgetting on HellaSwag, while GRPO remained stable. As Countdown accuracy increased, ES HellaSwag performance plummeted.

4 Analysis of ES Instability

4.1 Weight Drift (Frobenius Norm)

Drift is measured by the Frobenius norm of the update: $\|\Delta\theta\|_F$. ES-trained models exhibited weight drift magnitudes roughly **87x to 1000x higher** than GRPO-trained models over the same number of steps.

4.2 Sparsity and Targeted Updates

- **GRPO Updates:** Highly sparse and targeted. Significant changes are concentrated in specific layers, preserving the bulk of the model's prior state.
- **ES Updates:** Broad and dense. Because ES "doesn't know" which parameters are relevant, it applies updates across the entire architecture, specifically impacting V (Value) and MLP matrices. We spent some time discussing why the value matrices see more change than the query or key matrices.
- **Exception:** LayerNorm parameters in ES showed higher sparsity (fewer updates), leading to group discussion on whether preserving LayerNorm is a critical reason ES models maintain any coherence at all.

4.3 Critical Discussion / Open Questions

- Whether HellaSwag is a valid "prior task" for models it wasn't explicitly pre-trained on.
- Alternate ways to measure the weight update besides the Frobenius norm.
- Whether ES performance would stabilize if runs were 10x longer, experiments were repeated many times, or specific weights were frozen.